minicut Video Editor

Embedded JavaScript & Plugin API Reference Manual
Duktape 2.7.0 Engine Full UTF-8 / Multi-lingual Dockable Tool Panels Tool Shelf Dock Network Library (libcurl)

1. Introduction & JavaScript Architecture

minicut embeds a complete **JavaScript** engine (Duktape 2.7.0) with native UTF-8 and Unicode string support. JavaScript plugins provide full parity with the minicut scripting API, making it easy to create internationalized tools in Japanese, Arabic, and other languages.

Script Directory Locations

Location	Trigger / Role	Description
`~/.config/minicut/autorun/`	Startup / Auto-register	`.js` scripts execute on startup and add an icon button on the horizontal Tool Shelf Dock below the menu bar.
`~/.config/minicut/scripts/`	Menu Click / On Demand	`.js` scripts appear in the top-level **Scripts** menu on the main menu bar.
`~/.config/minicut/icons/`	Dock Icon Asset	32x32 PNG icon images matching script stems (e.g., `text_animator.png` for `text_animator.js`).

2. JavaScript Plugin Architecture

```
// ============================================================================
// Example JavaScript Plugin: custom_animator.js
// ============================================================================

var targetTrack = 0;
var textContent = "日本語テキスト字幕";

function applyAnimation() {
    minicut.push_undo();
    var clipId = timeline.place_text_clip(targetTrack, textContent,
timeline.get_playhead(), 5.0);
    if (clipId) {
        clip.set_position(clipId, 0.0, 150.0);
        clip.set_scale(clipId, 1.2);
        timeline.add_keyframe(clipId, "opacity", 0.0, 0.0);
        timeline.add_keyframe(clipId, "opacity", 0.5, 1.0);
        minicut.set_status("Placed text clip: " + textContent);
    }
}

plugin.init_panel = function() {
    gui.text("JavaScript Text Animator");
    gui.separator();
    targetTrack = gui.slider_int("Track Lane", targetTrack, 0, 4);
    textContent = gui.input_text("Caption Text", textContent);

    if (gui.button("Place Animated Text", applyAnimation)) {
        // inline logic
```

```
    }
};
```

3. Core JavaScript Modules

3.1. Project & File Automation (`project` / Global)

Function	Returns	Description
`project.save([path])` `project_save([path])`	`boolean`	Saves current project state to `.minicut` file.
`project.load(path)` `project_load(path)`	`boolean`	Loads a project from `path`.
`project.get_name()`	`string`	Returns the current project name.
`project.set_name(name)`	`void`	Sets current project name.
`project.import_video(path)`	`string`	Imports video into the project media asset pool.
`project.import_audio(path)`	`string`	Imports audio into the project media asset pool.
`project.import_image(path)`	`string`	Imports static image into media asset pool.
`assets.create_text(text, [opts])`	`string`	Creates text asset with font, outline, and alignment.

3.2. Network & Internet Library (`net`)

Function	Returns	Description
`net.get(url)`	`object`	HTTP GET request. Returns `{{status, body, ok, error}}`.
`net.post(url, body)`	`object`	HTTP POST request with string or JSON data.
`net.download(url, dest)`	`boolean`	Downloads remote asset directly to file.
`net.upload(url, path)`	`object`	Uploads file binary stream to remote URL.
`net.request(options)`	`object`	Custom HTTP request with method, body, timeout.

3.3. Timeline & Track Management (`timeline`)

Function	Returns	Description
`timeline.get_playhead()`	`number`	Current playhead position in

Function	Returns	Description
		seconds.
`timeline.set_playhead(t)`	`void`	Moves playhead to `t` seconds.
`timeline.get_duration()`	`number`	Total duration of timeline in seconds.
`timeline.get_track_count()`	`number`	Total count of tracks.
`timeline.add_track(type, [name])`	`number`	Adds a track ("video" or "audio") and returns index.
`timeline.place_video(trk, id, start, dur)`	`string`	Places video clip and returns clip ID.
`timeline.place_audio(trk, id, start, dur)`	`string`	Places audio clip and returns clip ID.
`timeline.place_image(trk, id, start, dur)`	`string`	Places image clip and returns clip ID.
`timeline.place_text_clip(trk, text, start, dur)`	`string`	Generates and places text clip on track.
`timeline.delete_clip(clip_id)`	`boolean`	Deletes clip and keyframes.
`timeline.split_clip(clip_id, [t])`	`boolean`	Splits clip at time `t` (or playhead).
`timeline.select_clips(ids_array)`	`void`	Selects multiple clips by ID array.
`timeline.get_selected_clips()`	`Array<string>`	Returns array of selected clip IDs.
`timeline.clear_selection()`	`void`	Clears clip selection.
`timeline.move_track_up(idx)`	`void`	Moves track up in stack order.
`timeline.move_track_down(idx)`	`void`	Moves track down in stack order.
`timeline.set_track_name(idx, name)`	`void`	Renames track at index.
`timeline.get_track_name(idx)`	`string`	Gets track name at index.
`timeline.add_keyframe(id, prop, t, val)`	`void`	Adds keyframe for property at time `t`.

Function	Returns	Description
`timeline.remove_keyframe(id, prop, t)`	`void`	Removes keyframe near timestamp `t`.
`timeline.evaluate_keyframe(id, prop, t, [def])`	`number`	Calculates interpolated value at time `t`.

3.4. Clip Transformations (`clip`)

Function	Returns	Description
`clip.get_position(clip_id)`	`{{x, y}}`	Returns center offset object `{{x: 0, y: 0}}`.
`clip.set_position(clip_id, x, y)`	`void`	Sets center position offset in pixels.
`clip.get_scale(clip_id)`	`number`	Returns scale multiplier (1.0 = 100%).
`clip.set_scale(clip_id, s)`	`void`	Sets uniform scale multiplier.
`clip.get_rotation(clip_id)`	`number`	Returns rotation in degrees.
`clip.set_rotation(clip_id, r)`	`void`	Sets rotation in degrees.
`clip.get_opacity(clip_id)`	`number`	Returns opacity multiplier [0.0, 1.0].
`clip.set_opacity(clip_id, o)`	`void`	Sets opacity multiplier.
`clip.get_volume(clip_id)`	`number`	Returns volume multiplier [0.0, 2.0].
`clip.set_volume(clip_id, v)`	`void`	Sets volume multiplier.
`clip.get_time(clip_id)`	`{{start_time, duration, offset}}`	Returns timing object.
`clip.set_time(clip_id, start, dur, [off])`	`void`	Updates clip timing and media offset.
`clip.get_info(clip_id)`	`object`	Returns comprehensive metadata dictionary.

3.5. GUI UI Elements (`gui`)

Function	Returns	Description
`gui.text(text)`	`void`	Displays text string.

Function	Returns	Description
`gui.text_colored(r, g, b, a, text)`	`void`	Displays RGBA colored text.
`gui.button(label, [callback])`	`boolean`	Renders push button and calls callback if clicked.
`gui.slider_float(label, val, min, max, [cb])`	`number`	Renders float slider.
`gui.slider_int(label, val, min, max, [cb])`	`number`	Renders integer slider.
`gui.input_text(label, text, [cb])`	`string`	Single-line text entry box.
`gui.checkbox(label, state, [cb])`	`boolean`	Toggle checkbox.
`gui.same_line()`	`void`	Places next widget horizontally on same line.
`gui.separator()`	`void`	Draws horizontal divider.
`gui.file_dialog(options)`	`void`	Opens file dialog matching user settings.

3.6. Engine Utilities (`minicut` / `console`)

Function	Returns	Description
`minicut.push_undo()`	`void`	Pushes undo state.
`minicut.set_status(msg, [dur])`	`void`	Shows toast notification.
`minicut.log(...)` `console.log(...)`	`void`	Outputs debug log to terminal.